Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Deep Learning Architectures 2: CNNs

Harnessing Structures

- MLPs do not exploit the relation between the features (aka the prior knowledge)
- As a result, they grow very rapidly in size by the input dimension. For instance, 10^8 parameters for an MLP with 100 neurons processing 1 mega pixel image.
- Images, and other type of data, typically have two important structures
  - Translation Invariance: Moving an object does not change the object
  - Locality: Nearby pixels are related
- Turns out using convolution/cross-correlation could help leveraging these structures and reduce the number of parameters drastically.



Cat



Cat



- Consider the pre-activation result of the i-th neuron

$$v_i = \sum_j W_{ij} x_j + b_i = \sum_k W_{i(i+k)} x_{i+k} + b_i = \sum_k \omega_i^k x_{i+k} + b_i \tag{1}$$

- Translation invariance implies that a shift in the input should simply lead to a shift in the output
- This is only possible if ω and b are independent of specific location i:

$$v_i = \sum_k \omega^k x_{i+k} + b \tag{2}$$

effectively a convolution/cross-correlation

CNN: Example 1

- Let $x \in \mathbb{R}^5$ and $v \in \mathbb{R}^3$ such that $v = Wx + b$ has $3 + 3 \times 5 = 18$ learnable parameters in a regular MLP.
- With CNN, the number of parameters will reduce to $3 + 1 = 4$

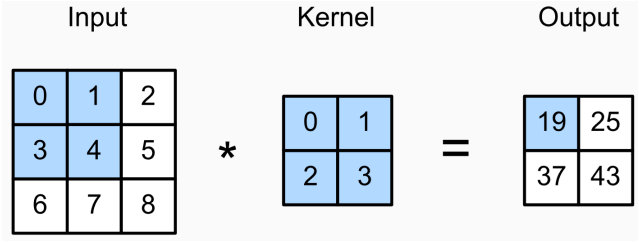
$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} \bullet & \bullet & \bullet & 0 & 0 \\ 0 & \bullet & \bullet & \bullet & 0 \\ 0 & 0 & \bullet & \bullet & \bullet \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix} \quad \text{i.e. } v_i = b + \sum_{k=-1}^{1} \omega_k x_{i+k},\ i = 1, 2, 3 \qquad (3)$$

each color denoting a learnable parameters

- Thus, we are doing weight sharing and zero-ing out some other weights
- Convolution filter/kernel: $\bullet\bullet\bullet$, output called a feature map
- Note that in CNNs, the size of input, output, and v are constrained:

$$d_{out} = d_{in} - d_\omega + 1 \qquad (4)$$

CNN: Example 2

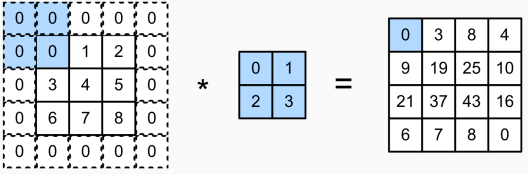


CNN Operations

- Locality: information relevant to the i-th output is localized. Thus, limit the range of sum in the convolution

$$v_i = b + \sum_{k=-\mathcal{O}(d)}^{\mathcal{O}(d)} \omega_k x_{i+k} \Rightarrow v_i = b + \sum_{k=-\Delta}^{\Delta} \omega_k x_{i+k} \tag{5}$$

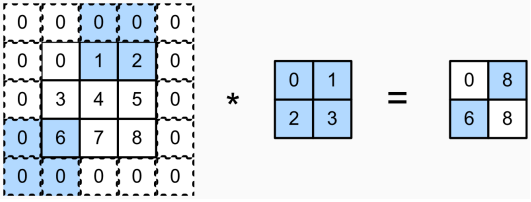
  where $\Delta = \mathcal{O}(1)$, e.g., $\Delta = 3, 5, 7, 9$ (typically odd). Thus, $d_\omega = 2\Delta + 1$
- Padding: Gaining control on output size and ensuring the preservation of edge information



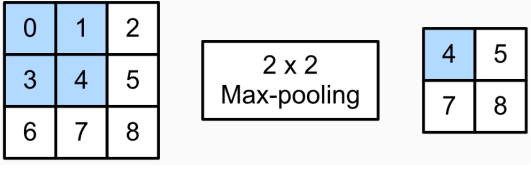
- Output size is $d_{in} - d_\omega + 1 + d_{padding}$
- $d_{padding} = d_\omega - 1$ ensures output has the same size as input.
- Δ to ensure even padding on each side

CNN Operations (Cont'd)

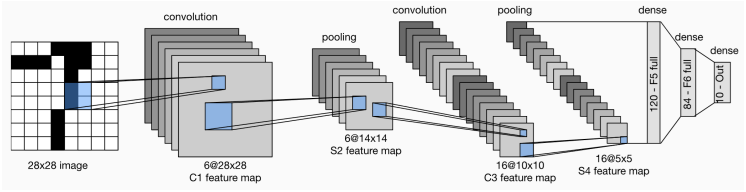
- Stride: More aggressive size reduction by shifting the kernel more than one location (say d_s location)

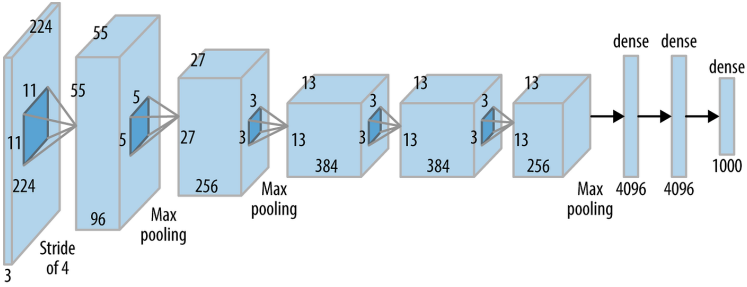


- Output size is $\frac{d_{in} - d_\omega + d_{padding} + d_s}{d_s}$
- Max and average pooling: Further parameter-free downsampling

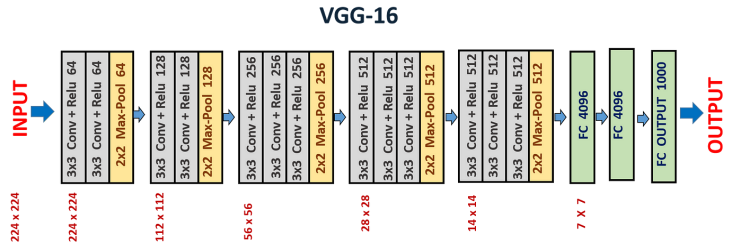


LeNet and AlexNet



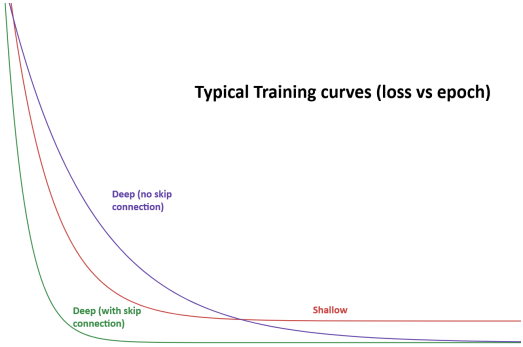


VGG and ResNets (Networks with Blocks)



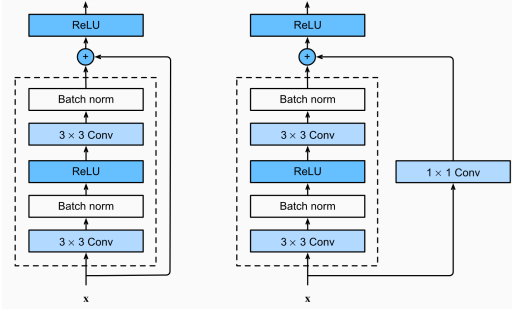
Residual/Skip Connection

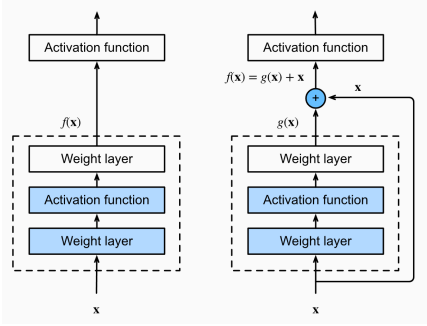
- We typically initialize NNs randomly
- In the early training as depth gets larger almost no useful signal goes from the input to the output and vice versa
- Residual connection helps carrying the information over while enjoying the high capacity of deep NNs



Residual Block

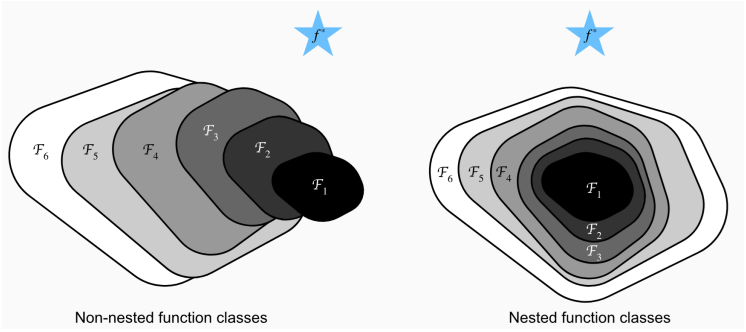
- Resnets constructed as a modular connection of Residual blocks
- Name: Each block learns the to model the residual $g(x)$ (difference of input and output). As the ideal residual is small, learning the residual might be easier than learning the true unknown feature-label relation $f(x)$
- Used in other architectures, e.g., transformers





Residual as Nested Function Classes

- The modularity leads to nested capacity as depth increases
- Thus, deeper models are highly expected to outperform shallower ones



Connection to ODEs



- Consider an ODE: $\frac{dx(t)}{dt} = f(t, x)$
- if f known, Euler's method can be used to model the evolution of x and solve the ODE: $x_{t+1} = x_t + f(t, x_t)$
- This is very similar to residual learning: res $x_{t+1} - x_t = f(t, x_t)$
- ResNets useful in solving and modeling ODEs and other physical governing equations, an area refereed to as Physics-informed DL